

1. ndarray (N-dimensional Array)

The `ndarray` is the fundamental object in NumPy. It's a grid of values, all of the same type, and is indexed by a tuple of non-negative integers.

Python

```
import numpy as np
```

```
# Create a 1-dimensional array (vector)
arr1d = np.array([1, 2, 3, 4, 5])
print("1D Array:\n", arr1d)
print("Type:", type(arr1d))
print("Number of dimensions (ndim):",
arr1d.ndim)
```

```
print("Shape (elements along each  
dimension):", arr1d.shape)  
print("Data type of elements (dtype):",  
arr1d.dtype)  
print("-" * 30)
```

```
# Create a 2-dimensional array (matrix)  
arr2d = np.array([[10, 20, 30], [40, 50,  
60]])  
print("2D Array:\n", arr2d)  
print("Number of dimensions (ndim):",  
arr2d.ndim)  
print("Shape (rows, columns):",  
arr2d.shape)  
print("Data type of elements (dtype):",  
arr2d.dtype)  
print("-" * 30)
```

```
# Create a 3-dimensional array (tensor)  
arr3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7,  
8]]])  
print("3D Array:\n", arr3d)  
print("Number of dimensions (ndim):",
```

```
arr3d.ndim)
print("Shape:", arr3d.shape)
print("-" * 30)
```

2. Array Creation

NumPy provides various functions to create arrays efficiently.

Python

```
# From Python lists (as shown above)
list_data = [1, 2, 3]
np_array_from_list = np.array(list_data)
```

```
# Zeros array
zeros_array = np.zeros((3, 4)) # 3 rows,
4 columns of zeros
print("Zeros Array:\n", zeros_array)
```

```
print("-" * 30)
```

```
# Ones array
```

```
ones_array = np.ones((2, 2)) # 2x2 array  
of ones
```

```
print("Ones Array:\n", ones_array)
```

```
print("-" * 30)
```

```
# Empty array (uninitialized, contains  
garbage values)
```

```
empty_array = np.empty((2, 3))
```

```
print("Empty Array (might contain  
garbage values):\n", empty_array)
```

```
print("-" * 30)
```

```
# Array with a range of numbers (like  
Python's range())
```

```
arange_array = np.arange(0, 10, 2) #  
Start at 0, stop before 10, step by 2
```

```
print("Arange Array:", arange_array)
```

```
print("-" * 30)
```

```
# Array with evenly spaced numbers
```

over an interval

```
linspace_array = np.linspace(0, 1, 5) # 5  
evenly spaced numbers between 0 and  
1 (inclusive)  
print("Linspace Array:", linspace_array)  
print("-" * 30)
```

```
# Identity matrix (square matrix with  
ones on the main diagonal)  
identity_matrix = np.eye(3) # 3x3 identity  
matrix  
print("Identity Matrix:\n", identity_matrix)  
print("-" * 30)
```

```
# Full array (filled with a specific value)  
full_array = np.full((2, 3), 7) # 2x3 array  
filled with 7  
print("Full Array:\n", full_array)  
print("-" * 30)
```

3. Indexing and Slicing

Accessing elements or sub-arrays from
ndarrays.

Python

```
arr = np.array([[10, 20, 30, 40],  
                [50, 60, 70, 80],  
                [90, 100, 110, 120]])  
print("Original Array:\n", arr)  
print("-" * 30)
```

Basic Indexing

```
print("Element at (0, 1):", arr[0, 1]) #  
Row 0, Column 1 (value 20)  
print("Element at (2, 3):", arr[2, 3]) #  
Row 2, Column 3 (value 120)  
print("-" * 30)
```

Slicing (similar to Python lists)

```
print("First row:", arr[0, :]) # All columns  
of the first row
```

```
print("First row (alternative):", arr[0]) # If  
only one dimension is sliced, it defaults  
to all of that dimension  
print("-" * 30)
```

```
print("First column:\n", arr[:, 0]) # All  
rows of the first column  
print("-" * 30)
```

```
print("Sub-array (rows 0-1, cols 1-2):\n",  
arr[0:2, 1:3])  
# This selects:  
# [20, 30]  
# [60, 70]  
print("-" * 30)
```

```
# Boolean Indexing (powerful for  
conditional selection)  
# Select all elements greater than 50  
print("Elements > 50:", arr[arr > 50])  
print("-" * 30)
```

```
# Select elements in the first row that
```

are greater than 20

```
print("First row elements > 20:", arr[0,  
arr[0, :] > 20]) # More complex: apply  
boolean mask to a specific slice  
print("-" * 30)
```

Fancy Indexing (using an array of indices)

Select specific rows

```
print("Selected rows (0 and 2):\n", arr[[0,  
2]])  
print("-" * 30)
```

Select specific elements using a list of (row, col) tuples

Not directly like this for multi-dimensional arrays, instead combine fancy indexing

e.g., to get (0,1) and (2,3)

```
rows = np.array([0, 2])
```

```
cols = np.array([1, 3])
```

```
print("Elements at (0,1) and (2,3):",  
arr[rows, cols])
```



```
print("-" * 30)
```

4. Data Types (dtype)

NumPy allows explicit specification of data types for memory efficiency and precision.

Python

```
int_array = np.array([1, 2, 3],  
dtype=np.int16)  
print("Int16 Array:", int_array, "dtype:",  
int_array.dtype)  
print("-" * 30)
```

```
float_array = np.array([1.0, 2.5, 3.7],  
dtype=np.float64)  
print("Float64 Array:", float_array,  
"dtype:", float_array.dtype)
```

```
print("-" * 30)
```

```
bool_array = np.array([True, False,  
True])
```

```
print("Boolean Array:", bool_array,  
"dtype:", bool_array.dtype)
```

```
print("-" * 30)
```

```
# Type conversion
```

```
converted_array =
```

```
int_array.astype(np.float32)
```

```
print("Converted to Float32:",
```

```
converted_array, "dtype:",
```

```
converted_array.dtype)
```

```
print("-" * 30)
```

5. Array Operations (Vectorization & Universal Functions)

The power of NumPy lies in its ability to perform operations on entire arrays without

explicit loops (vectorization), using highly optimized C implementations (ufuncs).

Python

```
arr_a = np.array([1, 2, 3])  
arr_b = np.array([4, 5, 6])
```

```
# Element-wise addition  
print("Addition:", arr_a + arr_b)  
print("-" * 30)
```

```
# Element-wise multiplication  
print("Multiplication:", arr_a * arr_b)  
print("-" * 30)
```

```
# Scalar operations  
print("Add scalar:", arr_a + 10)  
print("Multiply scalar:", arr_b * 2)  
print("-" * 30)
```

```
# Universal Functions (ufuncs)
print("Sine of arr_a:", np.sin(arr_a))
print("Square root of arr_b:",
np.sqrt(arr_b))
print("Exponential of arr_a:",
np.exp(arr_a))
print("-" * 30)
```

```
# Aggregation Functions
data = np.array([[1, 2, 3], [4, 5, 6]])
print("Original Data:\n", data)
print("Sum of all elements:",
np.sum(data))
print("Mean of all elements:",
np.mean(data))
print("Max of all elements:",
np.max(data))
print("-" * 30)
```

```
# Aggregation along an axis (axis=0 for
columns, axis=1 for rows)
print("Sum along columns (axis=0):",
np.sum(data, axis=0)) # Sums down the
```

```
columns: [1+4, 2+5, 3+6] = [5, 7, 9]
print("Mean along rows (axis=1):",
np.mean(data, axis=1)) # Mean across
rows: [(1+2+3)/3, (4+5+6)/3] = [2.0, 5.0]
print("-" * 30)
```

6. Broadcasting

Broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations.

Python

```
arr_2x3 = np.array([[1, 2, 3], [4, 5, 6]]) #
Shape (2, 3)
scalar = 10 # Shape
()
```

```
# Adding a scalar to an array
print("Array + Scalar:\n", arr_2x3 +
      scalar)
# Scalar 10 is "broadcast" to become
[[10,10,10], [10,10,10]] and then added
element-wise.
print("-" * 30)
```

```
vector_row = np.array([100, 200, 300]) #
Shape (3,)
# Adding a row vector to a 2D array
print("Array + Row Vector:\n", arr_2x3 +
      vector_row)
# Vector_row is broadcast along the
rows to match the 2D array's shape.
# [[1,2,3] + [100,200,300]]
# [[4,5,6] + [100,200,300]]
print("-" * 30)
```

```
vector_col = np.array([[10], [20]]) #
Shape (2, 1)
# Adding a column vector to a 2D array
print("Array + Column Vector:\n",
```

```
arr_2x3 + vector_col)
# Vector_col is broadcast along the
columns to match the 2D array's shape.
# [[1,2,3] + [10,10,10]]
# [[4,5,6] + [20,20,20]]
print("-" * 30)
```

7. Reshaping and Resizing

Changing the dimensions of an array.

Python

```
arr_1d = np.arange(12) # Array from 0 to
11
print("Original 1D Array:", arr_1d)
print("-" * 30)

# Reshape to 3 rows, 4 columns
```

```
arr_reshaped = arr_1d.reshape((3, 4))
print("Reshaped to (3, 4):\n",
arr_reshaped)
print("-" * 30)
```

```
# Reshape using -1 for one dimension
(NumPy infers the size)
arr_inferred_shape = arr_1d.reshape((4,
-1)) # 4 rows, NumPy figures out 3
columns
print("Reshaped to (4, -1):\n",
arr_inferred_shape)
print("-" * 30)
```

```
# Flatten an array (convert to 1D)
arr_flattened = arr_reshaped.flatten()
print("Flattened Array:", arr_flattened)
print("-" * 30)
```

```
# Transpose an array (rows become
columns and vice versa)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print("Original Matrix:\n", matrix)
```



```
print("Transposed Matrix:\n", matrix.T)  
print("-" * 30)
```

8. Stacking and Splitting

Combining and dividing arrays.

Python

```
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
arr3 = np.array([7, 8, 9])
```

```
# Horizontal Stack (columns are added)  
h_stack = np.hstack((arr1, arr2))  
print("Horizontal Stack:", h_stack)  
print("-" * 30)
```

```
# Vertical Stack (rows are added)
```

```
v_stack = np.vstack((arr1, arr2))  
print("Vertical Stack:\n", v_stack)  
print("-" * 30)
```

```
# Concatenate (more general stacking  
along a specified axis)
```

```
matrix1 = np.array([[1, 2], [3, 4]])  
matrix2 = np.array([[5, 6], [7, 8]])
```

```
# Concatenate along axis 0 (rows)
```

```
concat_rows = np.concatenate((matrix1,  
matrix2), axis=0)
```

```
print("Concatenate along axis 0:\n",  
concat_rows)
```

```
print("-" * 30)
```

```
# Concatenate along axis 1 (columns)
```

```
concat_cols = np.concatenate((matrix1,  
matrix2), axis=1)
```

```
print("Concatenate along axis 1:\n",  
concat_cols)
```

```
print("-" * 30)
```

```
# Splitting arrays
split_array = np.array([10, 20, 30, 40,
50, 60])
split_parts = np.split(split_array, 3) #
Split into 3 equal parts
print("Split Array into 3 parts:",
split_parts)
print("-" * 30)
```

```
matrix_to_split = np.array([[1, 2, 3], [4, 5,
6], [7, 8, 9]])
split_rows = np.vsplit(matrix_to_split, 3)
# Split vertically into 3 rows
print("Vertical Split:\n", split_rows)
print("-" * 30)
```

9. Linear Algebra

NumPy's `linalg` module provides optimized linear algebra operations.

Python

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
# Matrix Multiplication (dot product)
```

```
print("Matrix A:\n", A)
```

```
print("Matrix B:\n", B)
```

```
print("-" * 30)
```

```
# Using np.dot()
```

```
dot_product = np.dot(A, B)
```

```
print("Dot Product (np.dot):\n",
```

```
dot_product)
```

```
print("-" * 30)
```

```
# Using the @ operator (Python 3.5+)
```

```
at_operator_product = A @ B
```

```
print("Dot Product (@ operator):\n",
```

```
at_operator_product)
```

```
print("-" * 30)
```

```
# Transpose (already covered, but
important for linear algebra)
print("Transpose of A:\n", A.T)
print("-" * 30)
```

```
# Determinant of a matrix
det_A = np.linalg.det(A)
print("Determinant of A:", det_A)
print("-" * 30)
```

```
# Inverse of a matrix
inv_A = np.linalg.inv(A)
print("Inverse of A:\n", inv_A)
print("A @ inv_A (should be identity):\n",
A @ inv_A) # Close to identity due to
floating point precision
print("-" * 30)
```

```
# Eigenvalues and Eigenvectors
eigenvalues, eigenvectors =
np.linalg.eig(A)
print("Eigenvalues of A:", eigenvalues)
```

```
print("Eigenvectors of A:\n",  
eigenvectors)  
print("-" * 30)
```

10. Random Number Generation

The `np.random` module is crucial for simulations, statistical analysis, and machine learning.

Python

```
# Generate a single random float  
between 0.0 and 1.0  
random_float = np.random.rand()  
print("Random float:", random_float)  
print("-" * 30)
```

```
# Generate a 2x3 array of random floats
```

between 0.0 and 1.0

```
random_array = np.random.rand(2, 3)
```

```
print("2x3 Random Array (0-1):\n",
```

```
random_array)
```

```
print("-" * 30)
```

Generate random integers between a low (inclusive) and high (exclusive) value

```
random_integers = np.random.randint(1, 10, size=(2, 4)) # 2x4 array of integers
```

from 1 to 9

```
print("Random Integers (1-9):\n",
```

```
random_integers)
```

```
print("-" * 30)
```

Generate random numbers from a standard normal distribution (mean=0, std=1)

```
normal_dist_nums =
```

```
np.random.randn(3, 3)
```

```
print("Random numbers from standard normal distribution:\n",
```

```
normal_dist_nums)  
print("-" * 30)
```

```
# Set the random seed for reproducibility  
np.random.seed(42)  
print("Random numbers with seed 42  
(first call):", np.random.rand(3))  
np.random.seed(42) # Set the same  
seed again  
print("Random numbers with seed 42  
(second call, same result):",  
np.random.rand(3))  
print("-" * 30)
```

that NumPy's strength comes from its vectorized operations, which are significantly faster than equivalent Python loops for numerical tasks.